



Distributed Systems and Applications (DSA612S)

Assignment 2: Distributed Food Delivery Platform

Release Date: 20 July 2026 (Week 1)

Due Date: 05 October 2026 at 23:59 (Week 12)

Total Marks: 100

Submission Format: GitHub/GitLab Repository Link on eLearning

Question 1 : RESTful APIs — Library and Resource Management System (50 Marks)

1. Overview & Learning Objectives

This assignment requires you to design and implement a scalable, distributed **Food Delivery Platform** involving multiple independent actors (Restaurants, Customers, and Drivers). The goal is to apply real-world distributed systems skills including microservices architecture, event-driven communication using **Kafka**, persistent storage, and containerisation.

By the end of this assignment, students should be able to:

- Design and implement microservices with clear functional boundaries using **Ballerina**.
- Apply event-driven design using **Kafka topics** for asynchronous process coordination.
- Persist complex relational and event data in a **NoSQL (Redis, MongoDB) or SQL** database.
- Containerise and orchestrate a multi-service environment using **Docker Compose** or **Kubernetes**.

2. Problem Description

Modern food delivery involves a complex coordination of independent actors. A seamless experience requires that when a customer places an order, the restaurant is notified immediately, payments are processed securely, and drivers are dispatched based on availability.

The Ministry of Industrialisation and Trade has requested a distributed platform to support local SMEs (Restaurants and Delivery Drivers). The system must handle high concurrency during peak meal times and remain fault-tolerant. Communication between services **must be**

event-driven using Kafka to ensure that order lifecycles and notifications are processed reliably and asynchronously.

3. Required Services

1. **Customer Service:** Manage user accounts, delivery addresses, and historical order data.
 2. **Restaurant Service:** Manage digital menus, real-time inventory, and kitchen opening hours.
 3. **Order Service:** Manage the central order state machine and lifecycle:
 - CREATED → CONFIRMED → PREPARING → READY → OUT_FOR_DELIVERY → DELIVERED (or CANCELLED).
 4. **Payment Service:** Simulate payment processing and emit confirmation events.
 5. **Delivery Service:** Coordinate driver assignments and track real-time delivery status.
 6. **Notification Service:** Disseminate multi-channel alerts to customers, restaurants, and drivers.
 7. **Admin Service:** Generate reports on restaurant statistics and delivery performance.
-

4. Key Technologies

- **Backend:** Ballerina for all microservice implementations.
 - **Messaging:** Kafka Topics (e.g.: *orders.created*, *payments.completed*, *delivery.assigned*, *delivery.completed*, etc).
 - **Persistence:** MongoDB or SQL for service-specific data.
 - **Infrastructure:** Docker for containerisation and Docker Compose for orchestration.
-

5. Evaluation Criteria

Component	Weight
Kafka Setup & Topic Management: Correct producer/consumer logic and topic partitioning.	15%
Database Setup & Schema Design: Effective data modelling for distributed actors.	10%
Microservices Implementation (Ballerina): Logic, lifecycle management, and API design.	50%
Docker Configuration & Orchestration: Service isolation and environment stability.	20%
Documentation & Presentation: Clear architecture diagrams and live defence.	5%

[Students who haven't pushed any code to the repository will not be given the opportunity to present and defend the assignment. More particularly, if a student's username does not appear in the commit log of the group repository, that student will be assumed not to have contributed to the project and thus be awarded the mark 0.]

6. Creativity & Extensions (Bonus)

Students can earn additional marks by implementing:

- **Driver Location Simulation:** Real-time coordinate updates on an overlay map.
- **Route Optimization:** Algorithms to calculate the fastest path between restaurant and customer.
- **Surge Pricing:** Dynamic pricing models based on high demand or low driver availability.
- **Complete UI:** A deployable Web or Mobile interface for customers/drivers.
- **Observability:** Integrated monitoring using Prometheus and Grafana.

Submission & Academic Integrity

- **Group Work:** This is a group assignment (4 - 8 members). All members must be contributors in the repository.
 - **Bonus Marks:** additional marks will be allocated for creative and innovative system designs.
 - **Plagiarism:** 100% AI-generated code will be awarded a zero. AI tools should only be used as a guide.
 - **Presentation:** Groups will be required to present and defend their solution to receive marks.
 - **Deadline:** Commits made after **05 October 2026, 23:59** will not be accepted.
-